

1. Введение

В рамках описания бизнес-процессов слой **Gateway + Verificator** рассматривается как единый механизм:

- приёма внешних запросов,
- проверки JWT-токена,
- проверки прав доступа,
- маршрутизации запроса в нужный микросервис

Ниже бизнес-процессы сгруппированы по функциональным областям:

1. аутентификация и профиль пользователя
 2. работа с данными НГУ
 3. работа с группами
 4. работа с заметками
 5. уведомления
 6. системные процессы
-

2. Аутентификация и профиль пользователя

2.1. Регистрация пользователя

Цель: создание аккаунта студента по университетской почте.

Участники: пользователь, Gateway, User Service.

Основной сценарий:

1. Пользователь открывает форму регистрации.
2. Клиент отправляет запрос `POST /auth/register` в Gateway.
3. Gateway передаёт запрос в User Service.
4. User Service проверяет корректность введенных данных.
5. User Service проверяет, что адрес электронной почты принадлежит домену `@g.nsu.ru`.
6. User Service проверяет, существует ли пользователь с таким email.
7. Если пользователь не найден, создается новая запись пользователя.
8. Создаются начальные пользовательские настройки.

9. Генерируются `access token` и `refresh token`.
10. Gateway возвращает клиенту результат регистрации и токены.

Результат: пользователь зарегистрирован в системе.

2.2. Вход пользователя в систему

Цель: аутентификация пользователя и выдача токенов доступа.

Участники: пользователь, Gateway, User Service.

Основной сценарий:

1. Пользователь вводит email и пароль.
2. Клиент отправляет запрос `POST /auth/login` в Gateway.
3. Gateway передаёт запрос в User Service.
4. User Service находит пользователя по email.
5. User Service проверяет корректность пароля.
6. При успешной проверке создаётся новая пользовательская сессия.
7. Генерируются `access token` и `refresh token`.
8. Gateway возвращает клиенту токены.

Результат: пользователь успешно входит в систему.

2.3. Обновление access token

Цель: продление сессии без повторного входа.

Участники: пользователь, Gateway, User Service.

Основной сценарий:

1. Клиент отправляет запрос `POST /auth/refresh` с `refresh token`.
2. Gateway передаёт запрос в User Service.
3. User Service проверяет валидность `refresh token`.
4. Если токен валиден и не отозван, создаётся новый `access token`.
5. При необходимости создаётся новый `refresh token`.
6. Gateway возвращает клиенту новые токены.

Результат: пользователь продолжает работу в системе без повторной авторизации.

2.4. Выход из системы

Цель: завершение пользовательской сессии.

Участники: пользователь, Gateway, User Service.

Основной сценарий:

1. Пользователь инициирует выход из системы.
2. Клиент отправляет запрос `POST /auth/logout` в Gateway.
3. Gateway передаёт запрос в User Service.
4. User Service находит текущую сессию или соответствующий `refresh token`.
5. Сессия помечается как завершённая либо `refresh token` отзывается.
6. Gateway возвращает успешный ответ.

Результат: текущая сессия пользователя завершена.

2.5. Получение данных текущего пользователя

Цель: просмотр профиля пользователя.

Участники: пользователь, Gateway, Verificator, User Service.

Основной сценарий:

1. Пользователь открывает раздел профиля.
2. Клиент отправляет запрос `GET /users/me` в Gateway с JWT.
3. Gateway выполняет проверку токена.
4. При необходимости выполняется проверка аутентификации через Verificator.
5. Gateway передаёт запрос в User Service.
6. User Service получает данные пользователя из базы данных.
7. Gateway возвращает профиль клиенту.

Результат: пользователь получает собственные данные.

2.6. Обновление настроек уведомлений

Цель: настройка типов уведомлений, которые пользователь хочет получать.

Участники: пользователь, Gateway, User Service.

Основной сценарий:

1. Пользователь открывает настройки уведомлений.
2. Клиент отправляет запрос `PATCH /users/me/notifications` в Gateway.
3. Gateway проверяет JWT.
4. Gateway передаёт запрос в User Service.
5. User Service находит пользователя.
6. User Service обновляет настройки уведомлений.
7. Gateway возвращает подтверждение.

Результат: пользовательские настройки уведомлений обновлены.

2.7. Привязка Telegram-аккаунта

Цель: связать профиль пользователя с Telegram для получения уведомлений и работы через бота.

Участники: пользователь, Telegram Bot Service, Gateway, User Service.

Основной сценарий:

1. Пользователь запускает Telegram-бота.
2. Бот предлагает выполнить привязку аккаунта.
3. Пользователь отправляет команду или код привязки.
4. Telegram Bot Service отправляет запрос через Gateway в User Service.
5. Gateway проверяет токен или код привязки.
6. User Service находит пользователя и сохраняет `telegram_id`.
7. User Service подтверждает успешную привязку.
8. Gateway возвращает результат в Telegram Bot Service.
9. Бот отправляет пользователю сообщение об успешной привязке.

Результат: Telegram ID сохранён в профиле пользователя.

3. Работа с данными НГУ

3.1. Ручное обновление данных из НГУ

Цель: загрузка актуального расписания и списка академических групп.

Участники: администратор, Frontend, Gateway, Verificator, NSU Integration Service, Groups Service, Notifier Service.

Основной сценарий:

1. Администратор в веб-панели нажимает кнопку обновления данных.
2. Frontend отправляет запрос `POST /integration/updates` в Gateway.
3. Gateway проверяет JWT администратора.
4. Gateway или Verificator проверяет административные права.
5. Gateway передаёт запрос в NSU Integration Service.
6. NSU Integration Service обращается к внешним источникам НГУ.
7. Сервис получает данные о расписании и группах.
8. Полученные данные нормализуются.
9. Локальный кэш расписания и групп обновляется.
10. При необходимости обновлённые официальные группы передаются в Groups Service.
11. NSU Integration Service формирует событие для Notifier Service.
12. Notifier Service при необходимости запускает рассылку уведомлений.
13. Gateway возвращает результат во Frontend.

Результат: данные НГУ обновлены в системе.

3.2. Получение актуального расписания

Цель: получить текущее расписание занятий.

Участники: пользователь или администратор, Gateway, NSU Integration Service.

Основной сценарий:

1. Клиент отправляет запрос `GET /integration/schedule` в Gateway.
2. Gateway проверяет токен, если доступ к операции защищён.
3. Gateway передаёт запрос в NSU Integration Service.
4. NSU Integration Service извлекает расписание из собственного хранилища.
5. Данные при необходимости фильтруются и преобразуются.
6. Gateway возвращает расписание клиенту.

Результат: клиент получает актуальное расписание.

3.3. Получение списка академических групп

Цель: получить официальный список учебных групп.

Участники: пользователь или администратор, Gateway, NSU Integration Service.

Основной сценарий:

1. Клиент отправляет запрос `GET /integration/groups` в Gateway.
2. Gateway передаёт запрос в NSU Integration Service.
3. NSU Integration Service извлекает список академических групп из кэша.
4. Gateway возвращает ответ клиенту.

Результат: клиент получает список групп.

3.4. Получение статуса последнего обновления

Цель: узнать, когда и с каким результатом в последний раз выполнялось обновление данных из НГУ.

Участники: администратор, Frontend, Gateway, NSU Integration Service.

Основной сценарий:

1. Администратор открывает раздел статуса обновлений.
2. Frontend отправляет запрос `GET /integration/updates/status` в Gateway.
3. Gateway проверяет токен администратора.
4. Gateway передаёт запрос в NSU Integration Service.
5. NSU Integration Service получает информацию о последнем запуске синхронизации.
6. Gateway возвращает статус во Frontend.

Результат: администратор получает информацию о последнем обновлении данных.

3.5. Настройка обновления расписания

Цель: изменить параметры запуска обновлений.

Участники: администратор, Frontend, Gateway, Verificator, NSU Integration Service.

Основной сценарий:

1. Администратор открывает настройки обновления расписания.
2. Frontend отправляет запрос `PUT /integration/schedule/` в Gateway.
3. Gateway проверяет токен администратора.
4. Gateway или Verificator проверяет административные права.
5. Gateway передаёт запрос в NSU Integration Service.
6. NSU Integration Service сохраняет параметры обновления.
7. Gateway возвращает результат во Frontend.

Результат: настройки обновления расписания изменены.

4. Работа с группами

4.1. Получение списка групп

Цель: просмотр доступных групп.

Участники: пользователь, Gateway, Groups Service.

Основной сценарий:

1. Клиент отправляет запрос `GET /groups` в Gateway.
2. Gateway при необходимости проверяет токен.
3. Gateway передаёт запрос в Groups Service.
4. Groups Service получает список групп из базы данных.
5. Gateway возвращает результат клиенту.

Результат: пользователь получает список групп.

4.2. Создание кастомной группы

Цель: создать пользовательскую группу для дисциплины, очереди на лабораторную или иной задачи.

Участники: пользователь, Gateway, Verificator, Groups Service.

Основной сценарий:

1. Пользователь нажимает кнопку создания группы.
2. Клиент отправляет запрос `POST /groups` в Gateway.

3. Gateway проверяет JWT.
4. Gateway или Verificator проверяет право на создание группы.
5. Gateway передаёт запрос в Groups Service.
6. Groups Service создаёт запись новой группы.
7. Создатель автоматически записывается как владелец или администратор.
8. Gateway возвращает данные созданной группы.

Результат: в системе появляется новая кастомная группа.

4.3. Получение информации о группе

Цель: просмотр информации о группе и её составе.

Участники: пользователь, Gateway, Verificator, Groups Service.

Основной сценарий:

1. Клиент отправляет запрос `GET /groups/{groupId}` в Gateway.
2. Gateway проверяет аутентификацию пользователя.
3. Gateway или Verificator проверяет право просмотра группы.
4. Gateway передаёт запрос в Groups Service.
5. Groups Service возвращает сведения о группе и, при необходимости, список участников.
6. Gateway возвращает результат клиенту.

Результат: пользователь получает информацию о выбранной группе.

4.4. Обновление данных группы

Цель: изменить название, описание или другие параметры группы.

Участники: администратор группы, Gateway, Verificator, Groups Service.

Основной сценарий:

1. Администратор редактирует данные группы.
2. Клиент отправляет запрос `PATCH /groups/{groupId}` в Gateway.
3. Gateway проверяет JWT.
4. Gateway или Verificator проверяет право управления группой.

5. Gateway передаёт запрос в Groups Service.
6. Groups Service обновляет данные группы в базе.
7. Gateway возвращает обновлённую информацию клиенту.

Результат: информация о группе изменена.

4.5. Удаление группы

Цель: удалить кастомную группу.

Участники: владелец или администратор группы, Gateway, Verificator, Groups Service.

Основной сценарий:

1. Пользователь инициирует удаление группы.
2. Клиент отправляет запрос `DELETE /groups/{groupId}` в Gateway.
3. Gateway проверяет JWT.
4. Gateway или Verificator проверяет право удаления.
5. Gateway передаёт запрос в Groups Service.
6. Groups Service удаляет группу либо помечает её как удалённую.
7. Gateway возвращает успешный ответ.

Результат: группа удалена.

4.6. Добавление участника в группу

Цель: включить пользователя в состав группы.

Участники: администратор группы, Gateway, Verificator, Groups Service, User Service.

Основной сценарий:

1. Администратор группы выбирает пользователя и нажимает кнопку добавления.
2. Клиент отправляет запрос `POST /groups/{groupId}/members` в Gateway.
3. Gateway проверяет JWT.
4. Gateway или Verificator проверяет право управления группой.
5. При необходимости Groups Service проверяет существование пользователя через User Service.
6. Groups Service добавляет пользователя в состав группы.

7. Gateway возвращает подтверждение клиенту.

Результат: пользователь добавлен в группу.

4.7. Удаление участника из группы

Цель: исключить пользователя из группы.

Участники: администратор группы, Gateway, Verificator, Groups Service.

Основной сценарий:

1. Клиент отправляет запрос `DELETE /groups/{groupId}/members/{userId}` в Gateway.
2. Gateway проверяет JWT.
3. Gateway или Verificator проверяет право удаления участника.
4. Gateway передаёт запрос в Groups Service.
5. Groups Service удаляет запись о членстве пользователя в группе.
6. Gateway возвращает подтверждение.

Результат: пользователь удалён из группы.

4.8. Изменение роли участника группы

Цель: назначить пользователю новую роль в группе.

Участники: администратор группы, Gateway, Verificator, Groups Service.

Основной сценарий:

1. Клиент отправляет запрос `PATCH /groups/{groupId}/members/{userId}` в Gateway.
2. Gateway проверяет JWT.
3. Gateway или Verificator проверяет право редактирования состава группы.
4. Gateway передаёт запрос в Groups Service.
5. Groups Service изменяет роль пользователя в группе.
6. Gateway возвращает обновлённые данные клиенту.

Результат: роль участника группы изменена.

4.9. Общий сбор по группе

Цель: выполнить массовую рассылку участникам выбранной группы.

Участники: пользователь с правом управления группой, Gateway, Verificator, Groups Service, Notifier Service.

Основной сценарий:

1. Пользователь инициирует общий сбор в интерфейсе группы.
2. Клиент отправляет запрос на соответствующую операцию через Gateway.
3. Gateway проверяет JWT.
4. Gateway или Verificator проверяет право пользователя на групповую рассылку.
5. Gateway передаёт запрос в Groups Service.
6. Groups Service формирует событие групповой рассылки.
7. Событие передаётся в Notifier Service.
8. Notifier Service получает список участников группы.
9. Notifier Service формирует сообщения и запускает отправку.
10. Результат операции возвращается пользователю.

Результат: участники группы получают уведомление.

5. Работа с заметками

5.1. Получение общих заметок по дисциплине

Цель: просмотр заметок, связанных с определённой дисциплиной.

Участники: пользователь, Gateway, Verificator, Notes Service.

Основной сценарий:

1. Клиент отправляет запрос `GET /disciplines/{disciplineId}/notes` в Gateway.
2. Gateway проверяет токен пользователя.
3. Gateway или Verificator проверяет право доступа к заметкам дисциплины.
4. Gateway передаёт запрос в Notes Service.
5. Notes Service извлекает заметки по `disciplineId` с учётом правил видимости.
6. Gateway возвращает список заметок клиенту.

Результат: пользователь получает общие заметки по дисциплине.

5.2. Получение личных заметок пользователя

Цель: просмотр личных заметок текущего пользователя.

Участники: пользователь, Gateway, Notes Service.

Основной сценарий:

1. Клиент отправляет запрос `GET /notes/me` в Gateway.
2. Gateway проверяет JWT.
3. Gateway передаёт запрос в Notes Service.
4. Notes Service извлекает заметки, привязанные к текущему пользователю.
5. Gateway возвращает результат клиенту.

Результат: пользователь получает список личных заметок.

5.3. Создание общей заметки по дисциплине

Цель: создать заметку, доступную нескольким участникам.

Участники: пользователь, Gateway, Verificator, Notes Service, Groups Service, NSU Integration Service, Notifier Service.

Основной сценарий:

1. Пользователь открывает раздел заметок по дисциплине.
2. Клиент отправляет запрос `POST /disciplines/{disciplineId}/notes` в Gateway.
3. Gateway проверяет JWT.
4. Gateway или Verificator проверяет право создания заметки.
5. Gateway передаёт запрос в Notes Service.
6. Notes Service при необходимости проверяет существование дисциплины через NSU Integration Service.
7. Notes Service определяет правила видимости и связанные группы, при необходимости взаимодействуя с Groups Service.
8. Notes Service сохраняет заметку в базе данных.
9. Notes Service создаёт первую версию заметки.
10. Notes Service формирует событие для Notifier Service.
11. Notifier Service при необходимости инициирует отправку уведомлений.
12. Gateway возвращает пользователю результат.

Результат: общая заметка создана.

5.4. Создание личной заметки

Цель: создать заметку, доступную только текущему пользователю.

Участники: пользователь, Gateway, Notes Service.

Основной сценарий:

1. Пользователь открывает раздел личных заметок.
2. Клиент отправляет запрос `POST /notes/me` в Gateway.
3. Gateway проверяет JWT.
4. Gateway передаёт запрос в Notes Service.
5. Notes Service создаёт заметку с привязкой к текущему пользователю.
6. Notes Service создаёт первую версию заметки.
7. Gateway возвращает результат.

Результат: личная заметка сохранена.

5.5. Редактирование заметки

Цель: обновить содержание заметки с сохранением истории.

Участники: пользователь, Gateway, Verificator, Notes Service, Notifier Service.

Основной сценарий:

1. Пользователь выбирает редактирование заметки.
2. Клиент отправляет запрос `PATCH /notes/{noteId}` в Gateway.
3. Gateway проверяет JWT.
4. Gateway или Verificator проверяет право редактирования.
5. Gateway передаёт запрос в Notes Service.
6. Notes Service получает текущую версию заметки.
7. Notes Service сохраняет изменения.
8. Notes Service создаёт новую запись в истории версий.
9. Если заметка общая, Notes Service формирует событие для Notifier Service.
10. Gateway возвращает обновлённую заметку клиенту.

Результат: заметка обновлена, история изменений сохранена.

5.6. Удаление заметки

Цель: удалить заметку.

Участники: пользователь, Gateway, Verificator, Notes Service.

Основной сценарий:

1. Пользователь инициирует удаление заметки.
2. Клиент отправляет запрос `DELETE /notes/{noteId}` в Gateway.
3. Gateway проверяет JWT.
4. Gateway или Verificator проверяет право удаления.
5. Gateway передаёт запрос в Notes Service.
6. Notes Service удаляет заметку либо помечает её как удалённую.
7. Gateway возвращает успешный ответ.

Результат: заметка удалена.

5.7. Получение истории версий заметки

Цель: просмотр истории изменений заметки.

Участники: пользователь, Gateway, Verificator, Notes Service.

Основной сценарий:

1. Клиент отправляет запрос `GET /notes/{noteId}/versions` в Gateway.
2. Gateway проверяет JWT.
3. Gateway или Verificator проверяет право доступа к истории.
4. Gateway передаёт запрос в Notes Service.
5. Notes Service извлекает историю версий заметки.
6. Gateway возвращает список версий клиенту.

Результат: пользователь получает историю изменений заметки.

6. Уведомления

6.1. Создание и отправка уведомления

Цель: доставка уведомления одному пользователю или группе пользователей.

Участники: сервис-источник, Notifier Service, Groups Service, User Service, Telegram Bot Service.

Основной сценарий:

1. Один из сервисов, например Notes Service, NSU Integration Service или Groups Service, создаёт событие.
2. Сервис-источник отправляет запрос в Notifier Service.
3. Notifier Service сохраняет событие уведомления.
4. Notifier Service определяет тип уведомления и выбирает шаблон.
5. Если уведомление адресовано группе, Notifier Service запрашивает список участников в Groups Service.
6. Notifier Service запрашивает в User Service данные получателей и их настройки уведомлений.
7. Для каждого получателя формируется текст сообщения.
8. Для каждого сообщения создаётся запись доставки.
9. Notifier Service отправляет сообщение в Telegram Bot Service.
10. Telegram Bot Service ставит сообщение в очередь или немедленно отправляет его в Telegram.
11. Telegram Bot Service возвращает статус доставки.
12. Notifier Service обновляет журнал отправки.

Результат: уведомление доставлено, история отправки сохранена.

6.2. Получение списка уведомлений

Цель: получить перечень созданных уведомлений.

Участники: администратор, Frontend, Gateway, Notifier Service.

Основной сценарий:

1. Администратор открывает раздел уведомлений.
2. Frontend отправляет запрос `GET /notifications` в Gateway.
3. Gateway проверяет JWT и административные права.
4. Gateway передаёт запрос в Notifier Service.
5. Notifier Service извлекает список уведомлений.
6. Gateway возвращает результат во Frontend.

Результат: администратор получает список уведомлений.

6.3. Получение отдельного уведомления

Цель: просмотр конкретного уведомления.

Участники: администратор, Frontend, Gateway, Notifier Service.

Основной сценарий:

1. Администратор выбирает уведомление.
2. Frontend отправляет запрос `GET /notifications/{id}` в Gateway.
3. Gateway проверяет JWT и права доступа.
4. Gateway передаёт запрос в Notifier Service.
5. Notifier Service извлекает данные уведомления.
6. Gateway возвращает информацию во Frontend.

Результат: администратор получает данные выбранного уведомления.

6.4. Просмотр истории отправленных уведомлений

Цель: просмотр журнала уведомлений.

Участники: администратор, Frontend, Gateway, Notifier Service.

Основной сценарий:

1. Администратор открывает журнал уведомлений.
2. Frontend отправляет запрос `GET /notifications/logs` в Gateway.
3. Gateway проверяет JWT администратора.
4. Gateway передаёт запрос в Notifier Service.
5. Notifier Service извлекает журнал отправок.
6. Gateway возвращает результат во Frontend.

Результат: администратор получает журнал отправленных уведомлений.

6.5. Просмотр статистики уведомлений

Цель: получение агрегированной статистики по уведомлениям.

Участники: администратор, Frontend, Gateway, Notifier Service.

Основной сценарий:

1. Администратор открывает страницу статистики.
2. Frontend отправляет запрос `GET /notifications/statistics` в Gateway.
3. Gateway проверяет JWT и права администратора.
4. Gateway передаёт запрос в Notifier Service.
5. Notifier Service рассчитывает статистику по событиям и доставкам.
6. Gateway возвращает результат во Frontend.
7. Frontend отображает статистику.

Результат: администратор получает статистику уведомлений.

6.6. Получение списка шаблонов уведомлений

Цель: просмотр доступных шаблонов сообщений.

Участники: администратор, Frontend, Gateway, Notifier Service.

Основной сценарий:

1. Администратор открывает раздел шаблонов.
2. Frontend отправляет запрос `GET /notifications/templates` в Gateway.
3. Gateway проверяет JWT и права администратора.
4. Gateway передаёт запрос в Notifier Service.
5. Notifier Service извлекает список шаблонов.
6. Gateway возвращает результат во Frontend.

Результат: администратор получает список шаблонов уведомлений.

6.7. Создание шаблона уведомления

Цель: добавить новый шаблон для типовых сообщений.

Участники: администратор, Frontend, Gateway, Notifier Service.

Основной сценарий:

1. Администратор вводит данные шаблона.
2. Frontend отправляет запрос `POST /notifications/templates` в Gateway.
3. Gateway проверяет JWT и административные права.

4. Gateway передаёт запрос в Notifier Service.
5. Notifier Service сохраняет шаблон в базе данных.
6. Gateway возвращает результат во Frontend.

Результат: новый шаблон добавлен в систему.

6.8. Редактирование шаблона уведомления

Цель: изменить существующий шаблон.

Участники: администратор, Frontend, Gateway, Notifier Service.

Основной сценарий:

1. Администратор редактирует шаблон.
2. Frontend отправляет запрос `PATCH /notifications/templates/{id}` в Gateway.
3. Gateway проверяет JWT и права администратора.
4. Gateway передаёт запрос в Notifier Service.
5. Notifier Service обновляет шаблон.
6. Gateway возвращает результат во Frontend.

Результат: шаблон уведомления обновлён.

6.9. Удаление шаблона уведомления

Цель: удалить или деактивировать шаблон.

Участники: администратор, Frontend, Gateway, Notifier Service.

Основной сценарий:

1. Администратор инициирует удаление шаблона.
2. Frontend отправляет запрос `DELETE /notifications/templates/{id}` в Gateway.
3. Gateway проверяет JWT и права администратора.
4. Gateway передаёт запрос в Notifier Service.
5. Notifier Service удаляет шаблон либо помечает его как неактивный.
6. Gateway возвращает результат во Frontend.

Результат: шаблон удалён или деактивирован.

7. Системные процессы

7.1. Проверка прав доступа

Цель: централизованная проверка возможности выполнения защищённой операции.

Участники: Gateway, Verificator, User Service, Groups Service, Notes Service.

Основной сценарий:

1. В Gateway поступает защищённый запрос.
2. Gateway извлекает сведения о пользователе и запрашиваемом действии.
3. Gateway инициирует проверку в Verificator Service.
4. Verificator Service проверяет аутентификацию пользователя.
5. При необходимости Verificator Service запрашивает дополнительный контекст:
 - в User Service — сведения о пользователе;
 - в Groups Service — участие в группе и роль;
 - в Notes Service — принадлежность заметки и правила доступа.
6. Verificator Service формирует решение: разрешить или запретить операцию.
7. Ответ возвращается в Gateway.
8. Gateway либо передаёт запрос в целевой сервис, либо возвращает ошибку доступа.

Результат: доступ к защищённым операциям контролируется централизованно.

7.2. Работа пользователя через Telegram-бот

Цель: обеспечить основной интерфейс взаимодействия студента с системой.

Участники: пользователь, Telegram Bot Service, Gateway, целевой микросервис.

Основной сценарий:

1. Пользователь отправляет команду или нажимает кнопку в Telegram.
2. Telegram Bot Service получает `update` от Telegram API.
3. Bot Service определяет тип команды и текущее состояние диалога.
4. Bot Service формирует внутренний запрос в Gateway.
5. Gateway проверяет аутентификацию пользователя и права доступа.
6. Gateway маршрутизирует запрос в нужный микросервис:
 - User Service — профиль, привязка аккаунта, настройки;

- `Notes Service` — личные и общие заметки;
- `Groups Service` — группы и участники;
- `NSU Integration Service` — расписание и список групп.

7. Целевой сервис выполняет операцию и возвращает ответ.
8. Gateway передаёт ответ в Telegram Bot Service.
9. Telegram Bot Service преобразует данные в сообщение, список кнопок или меню.
10. Пользователь получает результат в Telegram.

Результат: студент взаимодействует с системой через Telegram-бот.

7.3. Административная работа через веб-панель

Цель: обеспечить администраторам доступ к служебным функциям системы.

Участники: администратор, Frontend, Gateway, целевой микросервис.

Основной сценарий:

1. Администратор открывает веб-панель.
2. Frontend отправляет запросы в Gateway.
3. Gateway проверяет JWT и административные права.
4. Gateway направляет запросы в нужные сервисы:
 - `NSU Integration Service` — обновление расписания и групп;
 - `Notifier Service` — статистика, логи, шаблоны;
 - `Groups Service` — управление кастомными группами.
5. Целевой сервис выполняет операцию.
6. Gateway возвращает ответ во Frontend.
7. Frontend отображает результат администратору.

Результат: администратор управляет системой через веб-панель.
